

Q1. Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'

Answer:

I'd explain that query processing happens in three main stages: parsing and translation, optimization, and execution. The parser first checks the syntax and converts the SQL into a relational algebra expression. Then, the optimizer evaluates multiple execution plans by estimating the cost—like I/O and CPU usage—of different join orders and access paths, ultimately selecting the plan with the lowest estimated cost for execution.

Q2. Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'

Answer:

Heuristic optimization uses predefined rules, like performing selections as early as possible to reduce intermediate result sizes. Cost-based optimization, however, relies on database statistics to estimate the actual resource cost of various plans. I would use heuristic optimization for simple queries where rules are consistently effective, but cost-based is essential for complex queries where data distribution significantly impacts performance.

Q3. Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'

Answer:

In a real project, I'd implement connectivity by choosing a driver compatible with the DBMS, configuring a connection string, and using a connection pool for efficiency. JDBC is a Java-specific API that provides direct connectivity to various databases via Java drivers. In contrast, ODBC is a language-independent C-based standard that allows different applications to communicate with any DBMS using a common interface.

Q4. Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'

Answer:

ACID properties ensure reliable transactions. Atomicity ensures a bank transfer is all-or-nothing; Consistency keeps the total balance constant; Isolation prevents two simultaneous transfers from interfering; and Durability ensures the transaction persists after a crash. For example, in a ticket booking system, Atomicity is critical so you don't pay for a seat that wasn't actually reserved.

Q5. Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'

Answer:

Concurrency problems include dirty reads, where a transaction reads uncommitted data; lost updates, where one write overwrites another; and unrepeatable reads, where data changes between reads. Two-Phase Locking (2PL) solves these by requiring a transaction to acquire all necessary locks in a growing phase and release them in a shrinking phase, ensuring that no other transaction can modify data being used.

Q6. Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'

Answer:

A deadlock occurs when two or more transactions are waiting for locks held by each other, creating a cycle. I'd use a wait-for graph for detection, where an edge from T_i to T_j indicates T_i is waiting for T_j ; a cycle in this graph signals a deadlock. Prevention strategies include the Wait-Die scheme or the Wound-Wait scheme, which use timestamps to decide which transaction to abort.

Q7. Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'

Answer:

Immediate Update writes changes to the disk as they happen, requiring undo logs if a transaction fails. Deferred Update waits until a transaction commits before writing to the disk, meaning it only needs redo logs. I'd prefer Immediate Update for long-running transactions to avoid huge memory buffers, whereas Deferred Update is better for short transactions to simplify recovery.

Q8. Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'

Answer:

Shadow Paging maintains two page tables: a current one and a shadow one. During updates, changes are made to new blocks, and the current table is updated; the shadow table is only replaced upon commit. Its main advantage is that it avoids the need for expensive undo/redo logs. However, it leads to data fragmentation and is less efficient for high-concurrency environments than log-based recovery.

Q9. Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'

Answer:

A serializability schedule is one that produces the same result as some serial execution of the same transactions. To test for conflict serializability, I'd build a precedence graph where an edge exists from T_i to T_j if they have conflicting operations and T_i occurs first. If the resulting graph is acyclic, the schedule is conflict serializable; otherwise, it is not.

Q10. Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'

Answer:

Basic 2PL allows releasing locks any time after the growing phase. Strict 2PL holds all exclusive locks until commit, preventing dirty reads. Rigorous 2PL holds all locks—both shared and exclusive—until commit. Strict 2PL is most commonly used because it balances concurrency with recovery simplicity, effectively avoiding cascading rollbacks while allowing some shared access.